

# Problem Set 02

## OOP Design, Composition, Special Methods & Iterators

Computer Science & Programming (Python)  
SUPSI

### Instructions

- Duration: 3 hours (in-class).
- You may use your notes and slides.
- Write clear and readable code.
- When required, answer conceptual questions in comments.
- Unless explicitly stated, you may assume inputs are of the correct type, but you must validate values (e.g., non-negative, non-empty).

### Exercise 1 — OOP Design Reasoning

We want to model a `ShoppingCart` that contains multiple `Item` objects.

Answer the following questions **in plain English**:

1. What is the **responsibility** of the class `Item`? What does it represent?
2. What is the **responsibility** of the class `ShoppingCart`? What does it own?
3. List **at least 5 operations** that should belong to the public interface of `ShoppingCart`.
4. Which attributes should be **non-public** in both classes, and why?
5. Give one example of incorrect usage that your design must prevent.

## Exercise 2 — Composition: Item and ShoppingCart

Implement two classes: `Item` and `ShoppingCart`.

### Class `Item`

Represents a product inside the cart.

#### Attributes

- `name` (string)
- `unit_price` (float)
- `quantity` (int)

#### Validation (raise `ValueError`)

- `name` must be a non-empty string (after trimming spaces)
- `unit_price` must be numeric and strictly positive
- `quantity` must be an integer and strictly positive

#### Methods

- `subtotal()` returning `unit_price * quantity`

### Class `ShoppingCart`

Represents a collection of `Item` objects.

#### Requirements

- Store items in a non-public attribute (e.g., `_items`).
- Implement `add_item(item)` where `item` is an `Item`.
- If an item with the same `name` already exists, increase its quantity instead of adding a duplicate.
- Implement `remove_item(name)` (raise `ValueError` if not found).
- Implement `total()` returning the sum of all subtotals.

### Conceptual questions (answer in comments)

1. Why should `_items` not be public?
2. Why is using an `Item` class better than storing tuples?
3. Where should validation happen and why?

## Exercise 3 — Special (Magic) Methods

Extend both classes.

### Item

Implement:

- `__repr__()`
- `__eq__()` (compare name, unit\_price, quantity)

### ShoppingCart

Implement:

- `__len__()` so that `len(cart)` returns the number of distinct items
- `__repr__()`
- `__str__()` printing a receipt-like summary
- `__eq__()` comparing contained items

### Testing

Write a script demonstrating:

- `len(cart)`
- `print(cart)` and `repr(cart)`
- equality between two carts

## Exercise 4 — Iterables and Membership

Make `ShoppingCart` behave like a container.

Implement:

- `__iter__()` to allow iteration over items
- `__contains__()` so that:
  - `"Pasta" in cart` checks by name
  - `Item("Pasta", 2.5, 3) in cart` checks by equality

### Testing

1. Add at least 5 items.
2. Iterate and print each item's name and subtotal.
3. Test membership using both string and `Item`.
4. Show that adding the same item twice increases quantity.

## Exercise 5 — Operator Overloading: `Vector2D`

Implement a class `Vector2D` representing a 2D float vector  $(x, y)$ .

Support:

- Initialization
- `__repr__()`
- `__eq__()`
- Addition `+`
- Subtraction `-`
- Scalar multiplication `*`

Calling `len(v)` must raise a `TypeError`.

In comments, explain why `len()` is not meaningful for a vector.